

Governance-Aware Routing for Networked LLM Applications: A Kubernetes Gateway Control Plane for Cost, Declared Sovereignty, and Budget Trade-offs

Anonymous Authors

Abstract—Networked LLM applications increasingly route requests through gateway infrastructures to multiple model providers, creating operational trade-offs between cost, latency, quality, budget pressure, workload ownership, and declared data-residency constraints. We present a Kubernetes-native gateway control plane that makes these routing decisions explicit and auditable. The system represents providers, models, budgets, routing policies, reports, and declared residency constraints as Kubernetes resources; filters non-compliant candidates using declared policy metadata; and ranks feasible candidates using cost, quality, latency, reliability, and budget-pressure signals implemented by the operator and experiment engines. We evaluate the approach in a bounded two-provider testbed while separating measured, cached, and simulated evidence. In cached replays of previously collected real API calls, the policy reduces cost by 70.8% relative to an always-premium baseline under the evaluated workload mix and committed catalog prices; the absolute total is 0.011 EUR over 40 prompts, so it is evidence of routing behavior rather than deployment-scale savings. A 30-repetition uncached run confirms stable cost separation under the fixed matrix but also a measurable latency increase. On objective GSM8K/MMLU tasks, the same cost-aware policy lowers exact-match accuracy from 65.4% to 47.2%, showing that static quality priors are insufficient for correctness-critical workloads. Simulated policy scenarios prevent configured violations by construction under declared metadata while exposing cost and availability trade-offs. An ephemeral Kubernetes integration run verifies gateway telemetry, workload attribution, and shadow-AI evidence ingestion. The result is a reproducible control-plane framework for trade-off visibility and auditability in cloud-native LLM routing.

Index Terms—LLM gateways, cloud-native applications, Kubernetes operators, model routing, FinOps, declared data sovereignty, AI governance, policy-as-code, LLM evaluation.

1 INTRODUCTION

A TYPICAL ENTERPRISE LLM APPLICATION IS A NETWORKED SERVICE: user requests enter through an API gateway, are attributed to a Kubernetes workload, and are forwarded to one of several managed model endpoints. The routing choice affects cost, latency, declared data-residency constraints, team budgets, and auditability. This makes LLM routing a networked application governance problem rather than only a model-selection problem.

Three operational problems recur. First, model spend is often opaque: gateway logs expose requests, but teams need attribution by namespace, application, team, provider, and model. Second, declared data-residency and sensitive-data policies are rarely embedded in the routing plane; they are often checked after the fact, if at all. Third, budget pressure is operational: when a namespace is near a monthly cap, the platform should expose whether cheaper feasible models, stricter guardrails, or blocking are appropriate.

We study these problems in a bounded systems artifact: a Kubernetes operator and gateway-adjacent experiment harness for governance-aware LLM routing. The artifact does not claim legal certification of provider residency, broad deployment validation, or universal quality guarantees. Instead, it provides a reproducible framework for making

routing trade-offs visible and auditable.

1.0.0.1 Contributions.: This paper makes four contributions:

- 1) A Kubernetes-native gateway control-plane architecture for governance-aware LLM routing in networked applications, using CRDs, reconciliation loops, telemetry collectors, and workload-level attribution.
- 2) A declared-policy routing formulation that filters non-compliant model candidates before scoring feasible candidates by cost, expected quality, latency, reliability, and budget pressure.
- 3) A reproducible evidence methodology that separates **MEASURED**, **CACHED**, and **SIMULATED** evidence across cost, quality, latency, declared-policy compliance, budget behavior, and integration verification.
- 4) A methodological result showing that open-ended LLM-as-judge quality can be misleading for objective tasks: GSM8K/MMLU exact-match accuracy drops under aggressive cost routing, motivating premium pinning or task-type guardrails for correctness-critical workloads.

TABLE 1
Capability positioning. The comparison is scoped to capabilities evaluated or explicitly provided by the cited systems, not all possible extensions.

Capability	FrugalGPT	Hybrid	RouteLLM
Cost-aware model selection	✓	✓	✓
Kubernetes CRDs/status	✗	✗	✗
Workload attribution	✗	✗	✗
Declared-residency filtering	✗	✗	✗
Budget phase integration	✗	✗	✗
Evidence separation	✗	✗	✗

Algorithm 1: Governance-aware LLM routing

Input: request metadata, catalog, policy, budget, weight

Output: selected model or block decision

```

1 candidates <- catalog models allowed for workload
2 rejected <- apply declared-policy filter(candidates,
3   if candidates is empty: block and emit rejection reason
4   compute cost from token estimate or observed usage
5   compute quality prior/evidence signal
6   compute latency and reliability from telemetry if available
7   compute budget-pressure signal from budget phase/usage
8   normalize features and score feasible candidates
9   select best candidate subject to guardrails
10  emit decision trace, scores, and rejected candidates

```

2 RELATED WORK

2.1 LLM Routing and Cascades

FrugalGPT [1], Hybrid LLM [2], RouteLLM [3], AutoMix [4], and recent bandit or mixture routers [5], [6] reduce cost by selecting models based on predicted difficulty, confidence, or learned preference data. These methods are complementary to this work. Our focus is not to outperform learned routers; it is to expose Kubernetes-native governance constraints, workload attribution, budget state, and audit traces around the routing decision. The heuristic difficulty proxy in the artifact is not a reproduction of RouteLLM or Hybrid LLM and is not used as a main comparative claim.

2.2 Gateways, Operators, and Telemetry

Gateway layers such as Envoy AI Gateway [7], Envoy Gateway [8], and LiteLLM [9] provide operational entry points for model traffic. Kubernetes operators and CRDs [10], [11] provide a natural control-plane abstraction: desired state is declared, controllers reconcile status, and decisions are visible through Kubernetes APIs. OpenTelemetry [12] and eBPF systems such as Tetragon [13] provide the telemetry substrate for usage attribution and direct-egress evidence.

2.3 FinOps, Policy, and Evaluation

FinOps and Kubernetes cost-allocation systems [14], [15] motivate per-team spend visibility. AI governance frameworks and regulations [16], [17], [18], [19], [20], [21] motivate auditable controls, but they do not by themselves define gateway-level routing mechanisms. Evaluation work such as HELM [22], GSM8K [23], MMLU [24], G-Eval [25], and LLM-as-judge studies [26] motivates separating open-ended quality proxies from objective correctness.

3 SYSTEM DESIGN

3.1 Control-Plane Resources

The operator models gateway governance with Kubernetes custom resources: *AIProvider* declares provider metadata and pricing, *AIModel* declares model/provider association and workload ownership, *AIGateway* declares telemetry collection, *AI SovereigntyPolicy* declares allowed and forbidden zones plus sensitive-data rules, *AIBudgetPolicy* declares budget thresholds and optional guarded fallback, *AIRoutingPolicy* emits recommendations, *AI FinOpsReport* summarizes spend, findings, and routing scores, and *AIQualityGate* evaluates candidate quality from golden evidence.

Fig. 1. Algorithmic structure of the control-plane routing decision. Evidence: IMPLEMENTED.

The operator enforces declared technical routing policies; it does not independently certify provider legal compliance or contractual data residency. Provider zones are catalog metadata. A compliant decision therefore means “compliant with configured declared metadata,” not proof of actual data movement.

3.2 Routing Formulation

The control plane separates hard constraints from soft scoring. A request first defines a feasible set of candidate models. Declared-policy filtering removes candidates whose provider metadata violates the active policy. Remaining candidates are ranked by cost, quality, latency, reliability, and budget signals.

The experiment harness uses a transparent argmin score:

$$S(m) = \alpha C_m(1 + \epsilon B) + \beta Q_m + \gamma L_m + \delta R_m, \quad (1)$$

where C_m is cost normalized to the premium model, B is budget pressure, Q_m is expected quality loss relative to premium, L_m is latency prior penalty, and R_m is residual declared-policy risk after filtering. The default harness weights are $\alpha = 1.0$, $\beta = 1.5$, $\gamma = 0.3$, $\delta = 0.5$, and $\epsilon = 1.0$.

The operator runtime scoring engine is different and is reported as implementation status rather than conflated with the harness. It computes a higher-is-better score in $[0, 1]$ per namespace/application/model/provider:

$$0.40 \text{ CostScore} + 0.30 \text{ QualityScore} + 0.20 \text{ LatencyScore} + 0.10 \text{ ReliabilityScore} \quad (2)$$

Cost and latency use inverse min-max normalization over observed groups; quality is a static catalog tier prior; reliability is $1 - \text{errors}/\text{requests}$. Sovereignty is a hard gate: a non-compliant model receives score 0 regardless of soft terms.

3.3 Quality and Budget Engines

The quality subsystem has three layers. The pure *qualityengine* computes a 0–100 composite score from correctness, reliability, latency, semantic, and judge dimensions; missing weighted evidence yields *insufficient-data* rather than a fabricated score. The *qualitystats* package adds non-inferiority testing and hysteresis. The *qualityeval* pipeline runs golden prompts against the production gateway and serializes evidence for the pure engine. In this paper, the open-ended

TABLE 2
Evidence map used in the revised paper.

RQ	Main claim	Evidence	Scope/limitation
RQ1	Cost and routing distribution	CACHED + MEASURED N=30	Two-provider harness; catalog-dependent
RQ2	Open-ended judge-assessed quality	CACHED	Judge proxy only
RQ2b	Objective accuracy	CACHED	GSM8K/MMLU subset
RQ3	Latency and overhead	CACHED + MEASURED N=30	Provider variance not isolated
RQ4	Declared-policy trade-offs	SIMULATED	Declared meta-data, not legal certification
RQ5	Budget behavior	SIMULATED	One pressure scenario

quality result uses the experiment harness judge scores; the operator quality gate is implemented but not claimed as a main evaluated result.

The budget engine maps projected monthly spend to phases *WithinBudget*, *Warning*, *Critical*, and *Exceeded*. Actions are advisory unless the user explicitly configures guarded fallback actuation against Envoy AI Gateway routes. The reported budget result is a simulated pressure scenario over existing workload rows.

3.4 Telemetry and Shadow-AI Evidence

The Envoy AI Gateway collector reads OpenTelemetry Prometheus metrics for token usage and request duration, then attributes usage to Kubernetes namespaces and applications using request headers or AIModel catalog metadata. Shadow-AI evidence is gateway-independent: eBPF/Tetragon-derived egress observations are loaded from a ConfigMap and classified against the same declared-policy engine. This detects direct egress to known LLM endpoints; it does not itself block traffic.

4 METHODOLOGY

4.1 Evidence Labels

4.2 Workloads and Data

The main matrix is a compact coverage-oriented workload set, not a production trace. Synthetic prompts were selected deterministically to cover sensitive HR, document QA, developer assistance, and analytical-agent cases. Public benchmark subsets provide objective ground truth for accuracy analysis. All paths in the artifact are repository-relative; provider credentials and secrets are not included.

4.3 Strategies and Metrics

The main manuscript reports strategies whose evaluated rows do not rely on a stubbed model: B1 premium-static, B4 static namespace policy, B5 budget hard block, and B6 proposed policy. We exclude earlier B2/B3/B7 rows from

TABLE 3
Workload construction.

Dataset/workload	Source	Prompts	Task type	Used for
W1 HR chatbot	Author-created	10	sensitive chat	RQ1–RQ5
W2 RAG docs	Author-created	10	document QA	RQ1–RQ5
W3 dev assistant	Author-created	10	code/explanation	RQ1–RQ5
W4 analytical agent	Author-created	10	analytical QA	RQ1–RQ5
GSM8K subset	Public [23]	250	math exact match	RQ2b
MMLU subset	Public [24]	250	multiple choice	RQ2b

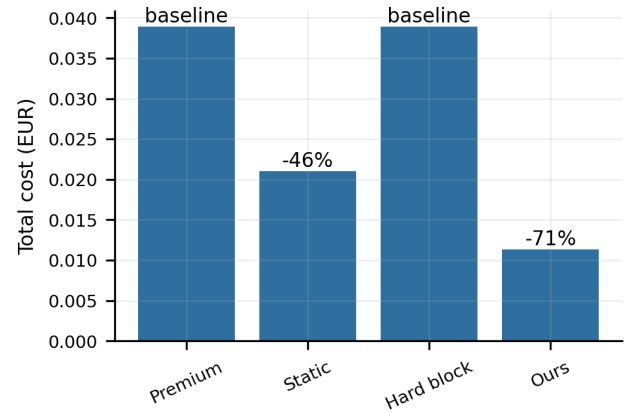


Fig. 2. CACHED: total replay cost over the 40-prompt workload matrix.

the main figures because some historical rows selected a modeled stub. Metrics are total cost, cost per request, cost per token, route distribution, open-ended judge score, exact-match accuracy, latency, configured-policy violations, served and blocked requests, and budget overrun.

The committed catalog prices are used as a reproducible price snapshot. Absolute cost values may change with provider pricing; relative conclusions are catalog-dependent.

5 RESULTS

5.1 Integration Verification: Telemetry and Attribution

An ephemeral Kubernetes integration run verifies that the operator can ingest Envoy AI Gateway telemetry, attribute token usage to workloads, create report status, and ingest shadow-AI direct-egress evidence. The integration run is used as systems verification only. It is not a production security evaluation and is not used to claim live prevention of bypass traffic.

5.2 RQ1: Cost and Routing Distribution

In the cached replay, **Ours** costs 0.011353 EUR over 40 prompts versus 0.038930 EUR for premium-static, a 70.8% reduction under the evaluated workload mix. Reported per 1,000 requests, this is 0.284 EUR for **Ours** versus 0.973 EUR for premium-static. The absolute total is small,

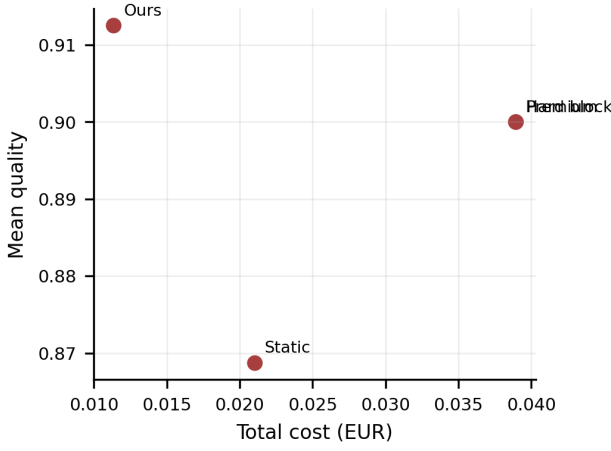


Fig. 3. CACHED: cost versus open-ended LLM-as-judge quality. The judge score is a proxy and should not be interpreted as objective correctness.

TABLE 4
CACHED: objective benchmark result and task-type guardrail recomposition.

Policy	Accuracy	Cost (EUR)	p95 latency
Premium-static	65.40%	0.116388	764 ms
Cost-aware	47.20%	0.007286	891 ms
Objective-premium guardrail	65.40%	0.116388	764 ms

so the result demonstrates that the control plane materially changes route selection under the catalog; it does not quantify production savings. The **Ours** route distribution is 30/40 gpt-4o-mini and 10/40 mistral-large; premium-static routes all 40 prompts to gpt-4o.

5.3 RQ2: Open-Ended Judge-Assessed Quality

Open-ended judge-assessed quality remains comparable in the evaluated prompt set: **Ours** has mean normalized score 0.9125, premium-static 0.9000, and pairwise win rate 50.0% over 40 comparisons. We do not interpret this as superiority or general quality guarantees. Inter-judge agreement is weak-to-moderate ($\kappa = 0.403$, Krippendorff $\alpha = 0.386$, $n = 120$), so judge quality is a bounded proxy.

5.4 RQ2b: Objective Accuracy and Failure Analysis

On 500 public GSM8K/MMLU items, **Ours** lowers cost by selecting cheaper models but reduces exact-match accuracy from 65.4% to 47.2%. This failure is central: static catalog quality priors and minimum-quality thresholds are insufficient for correctness-critical tasks. The simple guardrail “objective tasks use premium” restores the measured premium outcome by construction over the same items, at premium cost. Judge-vs-truth analysis over 300 objective items shows that 81.1% of incorrect answers are still rated acceptable (≥ 3), explaining why open-ended judge scores cannot replace exact-match evaluation.

5.5 RQ3: Latency Distribution and Routing Overhead

Cost-aware routing increases latency because it changes the provider/model mix. In the cached matrix, **Ours** has

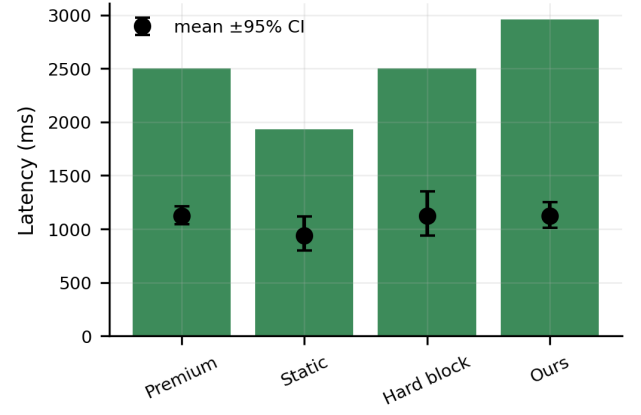


Fig. 4. CACHED: p95 latency bars and mean latency confidence intervals over the main workload rows.

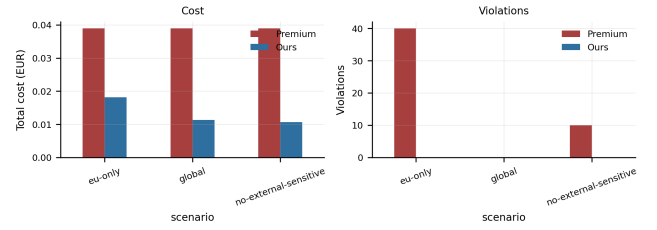


Fig. 5. SIMULATED: declared-policy scenarios over catalog metadata and cached routes. Zero configured-policy violations are expected from hard filtering; the scientific question is cost and availability impact.

mean latency 1509.95 ms and p95 3475 ms, compared with premium-static mean 1125.97 ms and p95 2494 ms. Routing-decision overhead remains small in the harness (10.83 microseconds on average for **Ours**), but end-to-end latency is operationally important.

5.6 RQ4: Cost, Availability, and Quality Under Declared Policy

Under declared metadata, policy filtering prevents configured violations by construction. In the EU-only scenario, premium-static violates the configured policy on 40/40 requests, while **Ours** serves 40/40 with zero configured-policy violations at 0.018160 EUR and mean quality 0.86625. In the no-external-sensitive scenario, premium-static violates 10 requests, while **Ours** serves 40/40 with zero configured-policy violations at 0.010691 EUR. EU zone is a declared provider/catalog attribute; the system does not verify actual data movement or legally certify residency.

5.7 RQ5: Budget-Pressure Behavior

In the simulated pressure scenario, alert-only serves 10/10 requests but exceeds the configured budget by 150%. Hard-block serves 6/10 and still exceeds the budget by 54.03%. The proposed graceful policy serves 10/10 with no overrun and mean quality 0.85. This is a scenario result, not a guarantee that fallback dominates in all deployments.

5.8 Sensitivity, Ablation, and Stability

The ablation shows that removing the cost term returns the policy to the premium-static cost (0.038930 EUR), while

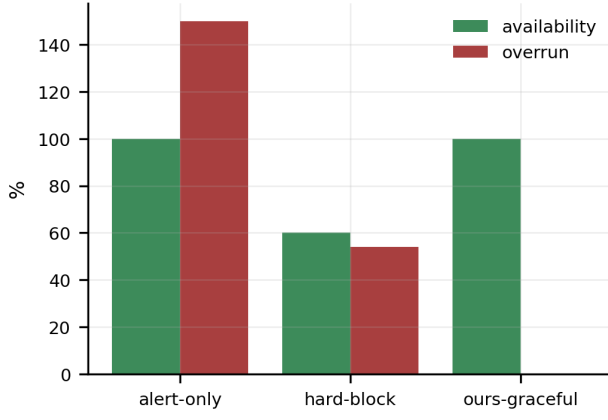


Fig. 6. SIMULATED: budget pressure on the analytical-agent workload.

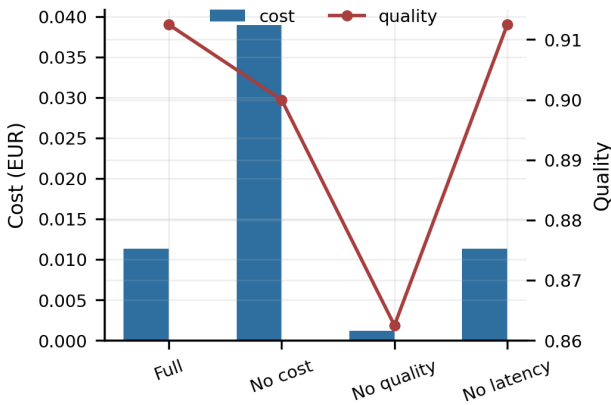


Fig. 7. CACHED/SIMULATED: ablation over routing score terms.

removing the quality term lowers cost to 0.001213 EUR but also lowers mean quality to 0.8625. Weight sensitivity is catalog-dependent: a policy-only least-cost variant estimates 0.003335 EUR but uses the cheapest model for all 40 prompts and has mean quality prior 0.8000.

Because the workload matrix and catalog prices are fixed, repeated cost measurements have low variance. Mann–Whitney tests and large effect sizes mainly confirm deterministic separation between policies under repeated executions. Practical interpretation should rely on effect size, route distribution, latency, quality, and prompt-level uncertainty.

Prompt-level bootstrap gives a wider uncertainty view. For **Ours**, total cost mean is 0.011401 EUR with 95% interval [0.005698, 0.018201], quality mean is 0.9124 [0.8375, 0.9688], and latency mean is 1512.23 ms [1200.28, 1850.13].

6 IMPLEMENTATION AND EVALUATION STATUS

7 DISCUSSION

7.1 Relationship to Existing Gateways and Learned Routers

LiteLLM and Envoy AI Gateway are complementary gateway substrates. Learned routers are also complementary: their difficulty predictors could supply a better quality

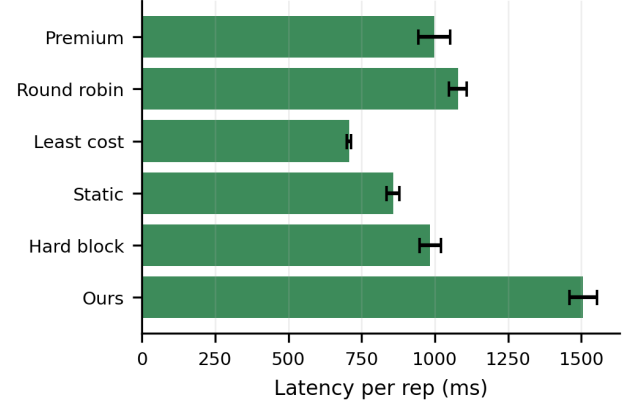
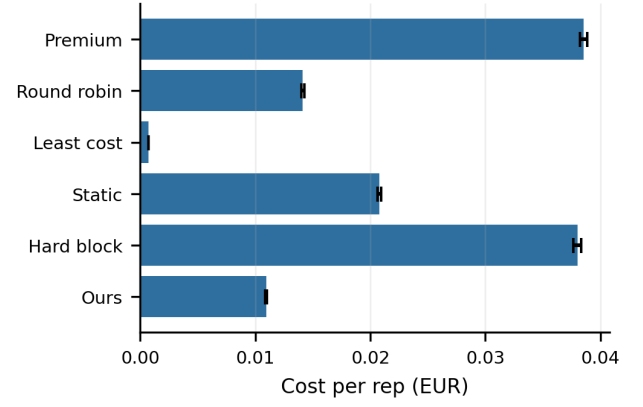


Fig. 8. MEASURED: 30 uncached repetitions over the fixed workload matrix. Statistics indicate run-to-run stability under fixed prompts and prices, not population-level enterprise variability.

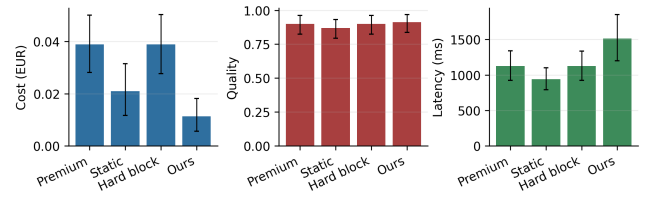


Fig. 9. CACHED: prompt-level bootstrap over the 40-item workload matrix.

signal inside the feasible set. The Kubernetes contribution is the governance control plane around routing: workload ownership, CRDs, reconciliation, namespace/application attribution, decision traces, budget phase, and declared-policy filtering.

7.2 Deployment Envelope and Policy Guardrails

The approach is most appropriate for multi-tenant, governance-constrained, mixed-difficulty workloads. Correctness-critical tasks should use premium pinning or strict task-type guardrails. Latency-sensitive paths should constrain candidates or increase latency weight. Single-tenant, single-provider, low-volume deployments may not need the added control-plane complexity. The weighted-sum policy is catalog-dependent and weight-sensitive; adaptive weights and constrained optimization are future work.

TABLE 5
Implementation/evaluation status.

Capability	Impl.	Live	Replay/sim.	Claim level
Cost attribution	✓	partial	✓	evaluated
Budget phase calculation	✓	partial	✓	evaluated scenario
Declared-residency filtering	✓	partial	✓	evaluated as declared policy
Routing score	✓	partial	✓	evaluated in harness
Envoy telemetry ingestion	✓	✓	✗	implemented in operator integration verified
Tetragon/shadow-AI evidence	✓	✓	✗	ingestion/detection verified
Budget fallback enforcement	✓	partial	✓	guarded, limited evidence
Production failover	partial	✗	✗	not claimed

8 THREATS TO VALIDITY

External validity. The evaluation uses a bounded two-provider testbed, four synthetic enterprise-style workloads, public GSM8K/MMLU subsets, and no production trace. Larger traces are needed to quantify savings under production request distributions.

Construct validity. Open-ended quality uses LLM judges with weak-to-moderate inter-judge agreement, and the premium model is both baseline and quality anchor in some analyses. Objective tasks are therefore evaluated with exact match and reported separately.

Internal validity. The 30-repetition run uses the same workload matrix; its statistics show stability under fixed prompts and prices, not independent population samples. Prompt-level bootstrap partially addresses item uncertainty but does not replace production traces.

Policy validity. Provider zone and sensitive-data suitability are declared catalog attributes. The operator filters and reports based on those attributes, but it does not verify actual provider data movement or legal compliance.

Systems validity. Integration evidence verifies telemetry attribution, quality job plumbing, and shadow-AI evidence ingestion in an ephemeral Kubernetes run. Scalability, multi-cluster behavior, and production failover are not stress-tested.

9 DATA AVAILABILITY AND REPRODUCIBILITY

The artifact repository contains the operator source code, Kubernetes manifests, experiment harness, datasets, cached responses, result files, and figure-generation scripts required to reproduce the reported analyses. Provider credentials and secrets are not included. All paths in the paper are repository-relative. The main analysis commands are:

```
cd experimentation
python3 scripts/analyze_results.py --results results
python3 scripts/bootstrap_items.py --calls results
python3 scripts/objective_guardrail.py
python3 scripts/analyze_stats.py
```

10 CONCLUSION

We presented a Kubernetes-native gateway control plane for governance-aware routing of networked LLM applications. The operator combines declared-residency filtering, workload attribution, budget state, cost, latency, reliability, and quality priors to produce auditable routing decisions and reports. In a bounded two-provider evaluation, the approach reduced cached replay cost by 70.8% relative to an always-premium policy, and a 30-repetition uncached run confirmed stable cost separation with a measurable latency increase. Simulated policy scenarios showed zero configured-policy violations under declared provider metadata by construction, with the main question being cost and availability under the feasible candidate set. Objective GSM8K/MMLU results showed that aggressive cost routing can reduce correctness, motivating premium pinning or task-type guardrails for correctness-critical workloads. Overall, the results support governance-aware LLM routing as a practical control-plane mechanism for making cost, policy, latency, and quality trade-offs explicit in cloud-native LLM applications. Future work will evaluate real self-hosted GPU serving and hosting economics.

REFERENCES

- [1] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," 2023. [Online]. Available: <https://arxiv.org/abs/2305.05176>
- [2] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. V. S. Lakshmanan, and A. H. Awadallah, "Hybrid LLM: Cost-efficient and quality-aware query routing," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.14618>
- [3] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," 2024. [Online]. Available: <https://arxiv.org/abs/2406.18665>
- [4] P. Aggarwal, A. Madaan, A. Anand, S. P. Potharaju, S. Mishra, P. Zhou, A. Gupta, D. Rajagopal, K. Kappaganthu, Y. Yang, S. Upadhyay, M. Faruqi, and Mausam, "AutoMix: Automatically mixing language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: <https://arxiv.org/abs/2310.12963>
- [5] Y. Li, "LLM Bandit: Cost-efficient LLM generation via preference-conditioned dynamic routing," 2025. [Online]. Available: <https://arxiv.org/abs/2502.02743>
- [6] X. Wang, Y. Liu, W. Cheng, X. Zhao, Z. Chen, W. Yu, Y. Fu, and H. Chen, "MixLLM: Dynamic routing in mixed large language models," 2025. [Online]. Available: <https://arxiv.org/abs/2502.18482>
- [7] Envoy Proxy Community, "Envoy AI gateway documentation," <https://aigateway.envoyproxy.io/docs/>, 2026, accessed 2026-06-13.
- [8] —, "Envoy gateway documentation," <https://gateway.envoyproxy.io/docs/>, 2026, accessed 2026-06-13.
- [9] BerriAI, "LiteLLM: Call 100+ LLM APIs in the OpenAI format," <https://github.com/BerriAI/litellm>, 2026, open-source LLM gateway/proxy. Accessed 2026-06-13.
- [10] Kubernetes Documentation, "Operator pattern," <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>, 2026, accessed 2026-06-13.
- [11] —, "Custom resources," <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>, 2026, accessed 2026-06-13.
- [12] OpenTelemetry Authors, "OpenTelemetry specification," <https://opentelemetry.io/docs/specs/otel/>, 2026, accessed 2026-06-13.
- [13] Cilium Tetragon Project, "Tetragon: eBPF-based security observability and runtime enforcement," <https://tetragon.io/>, 2026, accessed 2026-06-13.

- [14] FinOps Foundation, "Finops framework," <https://www.finops.org/framework/>, 2026, accessed 2026-06-13.
- [15] OpenCost Project, "Opencost: Open source cost monitoring for cloud native environments," <https://opencost.io/>, 2026, cNCF project. Accessed 2026-06-13.
- [16] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," NIST AI 100-1, 2023. [Online]. Available: <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10>
- [17] —, "Artificial intelligence risk management framework: Generative artificial intelligence profile," NIST AI 600-1, 2024. [Online]. Available: <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
- [18] OWASP Foundation, "OWASP top 10 for large language model applications 2025," <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2025, accessed 2026-06-13.
- [19] International Organization for Standardization, "ISO/IEC 42001:2023: Artificial intelligence management system," <https://www.iso.org/standard/42001>, 2023, accessed 2026-06-13.
- [20] European Union, "Regulation (EU) 2024/1689 of the european parliament and of the council (artificial intelligence act)," Official Journal of the European Union, 2024, published 12 July 2024; in force 1 August 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>.
- [21] —, "Regulation (EU) 2016/679 (general data protection regulation)," Official Journal of the European Union, 2016, <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [22] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar *et al.*, "Holistic evaluation of language models," 2022. [Online]. Available: <https://arxiv.org/abs/2211.09110>
- [23] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," 2021. [Online]. Available: <https://arxiv.org/abs/2110.14168>
- [24] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://arxiv.org/abs/2009.03300>
- [25] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2023, pp. 2511–2522. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.153/>
- [26] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.05685>